The Reading Agent

Tabular-conditioned active text reading for MulTaBench (Track 1)
Architecture and approach so far

NLP 097215 Final Project

Status: day 3 (2026-10-12)

Abstract

A MulTaBench row has numeric and categorical columns plus one or more free-text fields. We treat prediction as a sequential decision process. The agent first reads the *table*. It then decides, one piece at a time, which pieces of text are worth reading, and it can stop at any step. A table encoder initialises a stacked GRU. Each text piece is a (column, sentence-chunk) item embedded once with frozen E5-small-v2. An *anytime predictor* outputs $\hat{y}$ after any number of reads. An *expected-gain head* estimates how much each unread item would lower the loss, and the agent reads greedily while that estimate exceeds a cost λ. On women-clothing (fold 0), reading everything matches Joint Frozen. At tight budgets, the learned policy beats a random reader by +0.014–0.016 AUROC.

1 Big picture

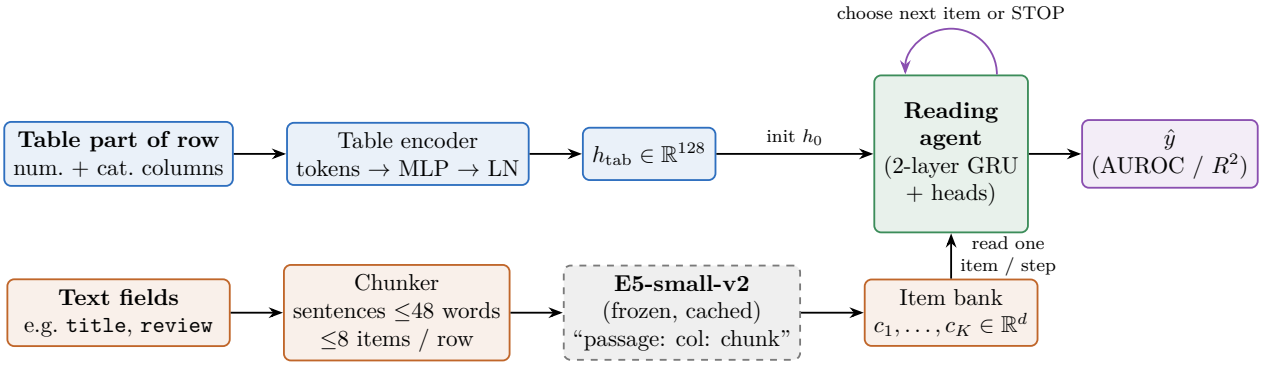


Figure 1: Overview. The table is always seen. Text is *acquired* item by item, and the policy decides how much of it to read.

The design follows course concepts. The encoder's output is the decoder's *initial hidden state* (seq2seq, L9). The model is a stacked GRU with inter-layer dropout (L8). Reading is an MDP with a per-step cost, which acts like a length penalty (L11). We tried imitation learning from an oracle with DAgger roll-ins (L11, T8) and then replaced it with value-of-information regression (§3).

2 Components

2.1 Items: what the agent can read

Each text field is split into sentences. Short sentences are merged greedily up to 48 words, and over-long sentences are cut into fixed 48-word windows. A short field gives a single item. Items are allocated round-robin across fields, up to $K \leq 8$ per row. Each item is verbalised as `"passage:`

`<column>: <chunk>"`, the same format MulTaBench uses. It is embedded *once* with frozen E5-small-v2 (384-d) and cached to disk. Inside the model, item i becomes

$$c_i = \mathrm{LN}\big(W_{\text{item}}\, e_i \;+\; W_{\text{name}}\, \mathrm{E5}(\text{field name}) \;+\; p_{\text{pos}(i)}\big) \in \mathbb{R}^{128}.$$

2.2 Table encoder → initial state

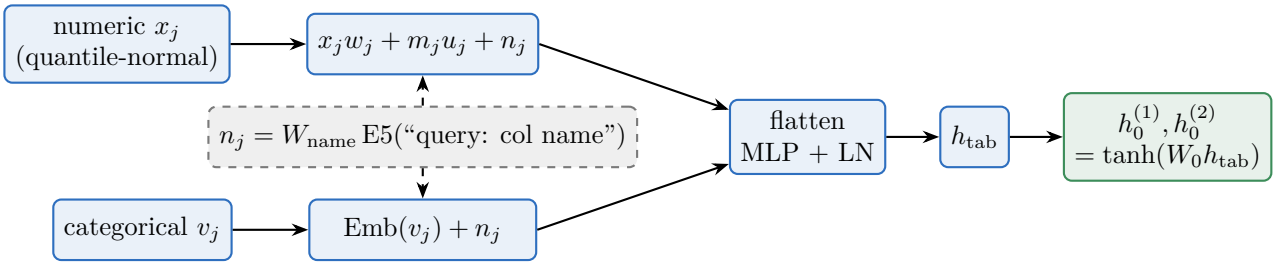


Figure 2: Table encoder. m_j is the missing indicator, and n_j is the projected column-name embedding. With no table columns, a learned null vector is used.

2.3 The read loop (unrolled)

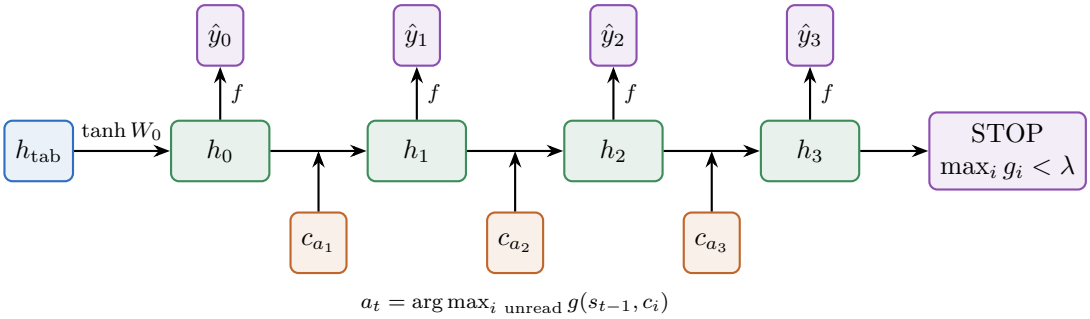


Figure 3: Unrolled reading. Each step feeds one chosen item into the 2-layer GRU. The predictor f can be queried after every step, so the model is an *anytime* predictor.

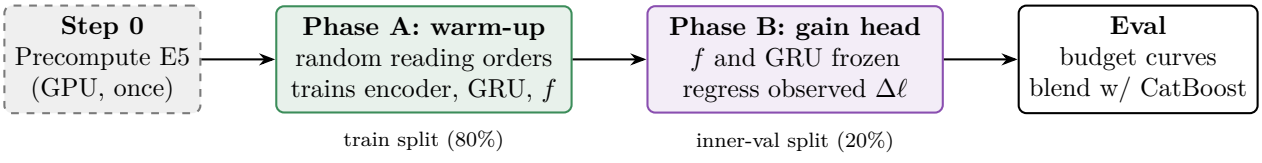
The state summary after t reads is

$$s_t = \big[\, h_t^{(2)} \;;\; \tfrac{1}{t}\textstyle\sum_{\tau \le t} c_{a_\tau} \;;\; h_{\text{tab}} \,\big] \in \mathbb{R}^{384},$$

and all heads read from it:

- **Predictor** $f(s_t)$: an MLP ($384 \to 256 \to n_{\text{out}}$, GELU, dropout).
- **Expected-gain head** $g(s_t, k_i) = \mathrm{MLP}([s_t; k_i])$, the predicted reduction in loss from reading item i next. The key k_i is either the item itself (*content* keys) or only its field and position (*meta* keys, a true feature-acquisition setting in which content is unseen before reading).

3 Training



Phase A (warm-up). For each batch, sample a random reading order and unroll over all items. The loss mixes every prefix with the full read:

$$\mathcal{L}_A = \tfrac{1}{2} \cdot \frac{1}{K+1} \sum_{t=0}^{K} \ell\big(f(s_t), y\big) \;+\; \tfrac{1}{2}\, \ell\big(f(s_K), y\big).$$

The loss ℓ is cross-entropy for classification and MSE for regression. We train with AdamW (lr 10^{-3}), clip gradients at norm 1, and early-stop on the val loss of the full read. Random orders make f reliable after *any* subset of reads, which the policy needs.

Phase B (expected-gain policy). With f frozen, roll in ε-greedily ($\varepsilon = 0.3$) using the current g. At every visited state, compute the observed one-step gain of every unread item i:

$$\Delta_i(s_t) = \ell\big(f(s_t), y\big) - \ell\big(f(s_t \oplus c_i), y\big), \qquad \mathcal{L}_B = \sum_{t,\, i \text{ unread}} \big(g(s_t, k_i) - \Delta_i(s_t)\big)^2.$$

The label is needed only to build the regression *targets*. At test time g estimates $\mathbb{E}[\Delta \mid s_t, k_i]$ without y, which is a label-free value-of-information estimate. The head is fit on the 20% inner-validation split because f's in-sample losses are biased low. Rolling in with the agent's own policy (DAgger-style) keeps the training states on the test-time distribution. **Inference:** read $\arg\max_i g$ while $\max_i g \geq \lambda$. Sweeping λ traces an accuracy-versus-budget curve.

Why not imitation? (negative result). Our first policy was a pointer network with a STOP key, trained to imitate a greedy *label* oracle. The oracle picks the item that most lowers the loss for the *true* y. Its choices depend on y, so a policy that cannot see y cannot imitate them. In practice the imitation policy did no better than random at every λ. The oracle's own AUROC of 0.966 with 1.5 reads reflects cherry-picking by label, so it is not an achievable upper bound. Regressing the *expected* gain removes this problem.

4 Evaluation setup and baselines

- MulTaBench text datasets (20), with the official repeated 3-fold $\times 2$ splits (seed 42) vendored in the code. Metrics are AUROC (macro OvR for multiclass) and R^2.
- **Agent modes:** table only ($t = 0$), read-all, gain@λ, random at a matched per-row budget, and the label oracle (diagnostic only).
- **Baselines:** CatBoost Joint-Frozen (num + cat + PCA-30 of whole-field E5 per text column), CatBoost tab-only, table-conditioned soft attention over all items, OOF stacking, and a val-tuned convex blend of CatBoost and the agent.
- **Bar to beat:** the method is target-aware, so the relevant bar is *Joint TAR*. We also report against Joint Frozen (official leaderboard CSVs, Wilcoxon over dataset $\times$ fold).

5 Results so far (women-clothing, fold 0)

Model	AUROC
CatBoost tab-only	0.7825
Agent, table only (no reads)	0.7826
Soft attention over all items	0.8976
Agent, read-all	0.8990
CatBoost Joint-Frozen (ours / leaderboard)	0.9024 / 0.9028
OOF stacking	0.9032
Val-tuned blend CatBoost + agent	**0.9046**
CatBoost Joint-TAR (leaderboard)	≈0.909

Table 1: Accuracy. Our CatBoost Frozen reproduces the leaderboard, which validates the pipeline.

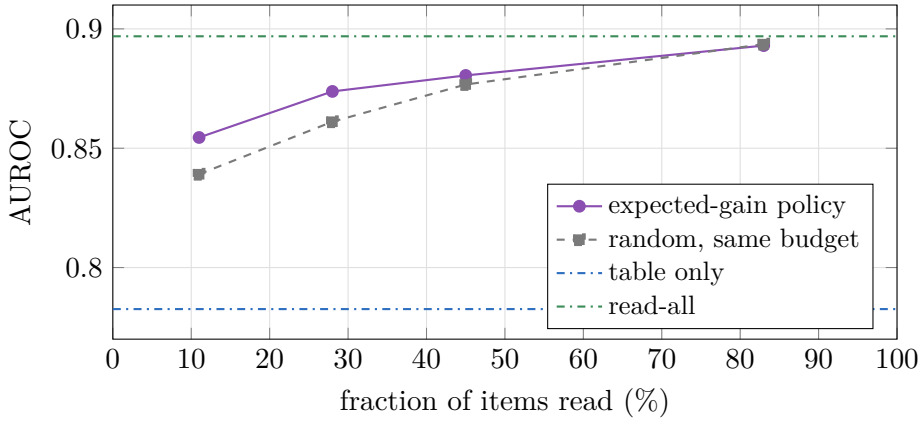


Figure 4: Accuracy versus reading budget (experiment #3, content keys). The learned policy's advantage grows as the budget shrinks (+0.016 at 11% of items read). Near full budget the advantage is unstable across initialisations. Meta keys stay close to random.

6 Iteration log (day 3, 2026-10-12)

Fixes. A static audit found a bug in the gain-head loss. The loss was normalised by a count that kept growing across the batches of an epoch, which silently decayed the learning rate within each epoch. It now uses a per-batch mean. Other fixes:

- Multiclass AUROC is computed over the classes present in the test set; before, a missing class gave NaN.
- CatBoost class names are fixed in advance.
- Each run has an id, and runs without a completion marker are ignored when results are aggregated.

Budget-targeted curves. A λ grid means different budgets on different datasets. We therefore bisect λ on the validation split to hit target read fractions of 10/25/50/75%. This needs only g, not labels.

Dataset (fold 0)	Variant	table only	read-all	CB Frozen	blend
fake-job	default	0.695	0.867	0.915	0.919
fake-job	+ field meta	**0.761**	0.858	0.915	0.920
fake-job	+ plateau lr	0.702	0.854	0.915	0.915
fake-job	whole-field items	0.687	0.863	–	–
women-clothing	default	0.783	0.899	0.902	0.905
women-clothing	+ plateau lr	0.783	0.897	0.902	0.904

Table 2: Ablations. "Field meta" gives the table encoder a presence flag and the log length of each text field.

Dataset	Variant	10%	25%	50%	75%
fake-job	default	+0.024	+0.015	+0.017	+0.004
fake-job	+ field meta	−0.005	−0.003	−0.003	−0.001
women-clothing	default	+0.013	+0.012	+0.006	+0.001

Table 3: AUROC advantage of the gain policy (content keys) over random reading at the same per-row budget.

Findings.

1. **The gain policy generalises** from women-clothing to fake-job, and its advantage is largest at tight budgets.
2. **Part of that advantage is metadata.** When the table encoder already knows which fields exist and how long they are, the policy's edge over random disappears on fake-job. A large share of the value of information there is "read the fields that exist / are long". This should be stated honestly in the paper.
3. **The read-all gap to CatBoost on fake-job (≈ 0.05) is in the learner, not the chunking.** Whole-field items, which are exactly the strings CatBoost embeds, give the same read-all AUROC. The warm-up validation loss is very noisy on this imbalanced dataset. A plateau LR schedule did not help.
4. **Fake-job varies by ± 0.01 from run to run**, so conclusions need the multi-dataset, multi-fold sweep.

7 Status and next steps

1. Embedding cache: 15 of 20 datasets are done (wine was added on day 3). Five remain: kickstarter, us-accidents, mercari, jigsaw and spotify.
2. Fold-0 sweep over the 15 cached datasets (recipe locked: no field meta, no schedule, budget curves). The first attempt was killed by the system's low-memory reaper.
3. Finish the gain-head ablation (val vs. train+val, 15 vs. 40 epochs) and lock the recipe.
4. Sweep 20 datasets $\times$ folds 0–2, then compare to the leaderboard with Wilcoxon tests.
5. Paper framing: *tabular-conditioned active text reading.* Headline results are the budget curves, the blend versus Frozen, and an honest comparison with TAR.